educación IT
CODE YOUR FUTURE

Bootcamp DevOps Engineer

Fase 1 - SysAdmin

Módulo 4

Compilación de aplicaciones

Introducción

La manera común de **instalar programas en Linux es mediante paquetes ya compilados** por la distribución. En casos excepcionales puede ser necesario compilar, algunas razones son:

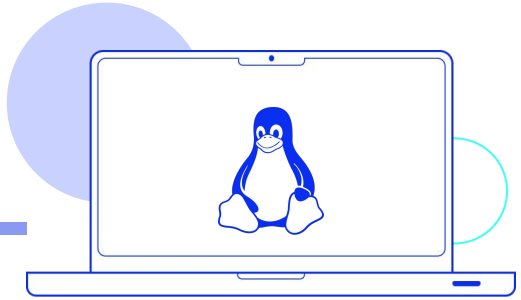
- El paquete no está en los repositorios de la distribución.
- El paquete está en los repositorios de la distribución pero está muy desactualizado.
- Queremos compilarlo quitando y/o agregando funcionalidades.

Los desarrolladores adoptan distintas maneras y *software* para permitir que sus programas sean instalados, algunos de ellos son:

- Archivo Makefile.
- Autotools.
- CMake.

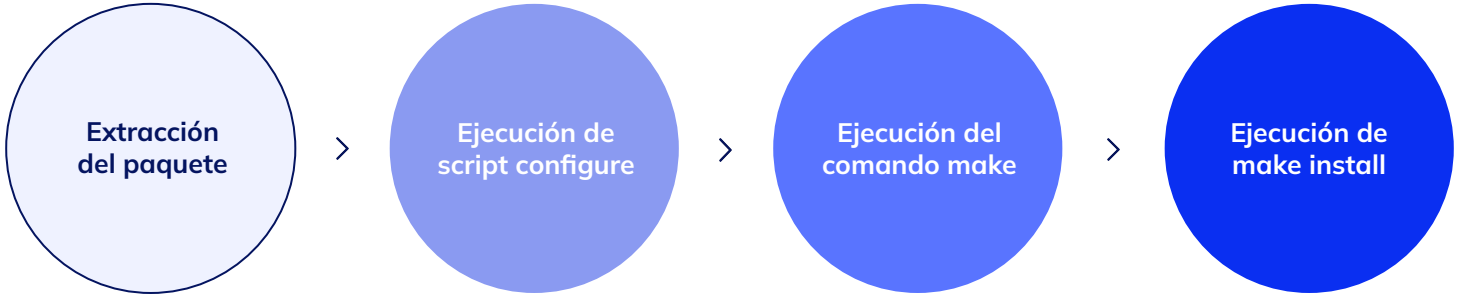
En general, los paquetes con código fuente vienen como archivos **`tarball`** (extensiones `.tar.gz`, `.tar.bz2`, `.tar.xz`) o sencillamente como `zip`.

La mayoría de estos paquetes se bajan desde sitios como GitHub, GitLab, o del propio proyecto. Es costumbre que este tipo de paquetes incluyan archivos llamados **`INSTALL`** y **`README`**. Estos archivos son muy importante leerlos ya que contienen información sumamente útil sobre los pasos a seguir para compilar y los requisitos previos para llevar a cabo dicha acción.



Compilar programas empaquetados con Autotools

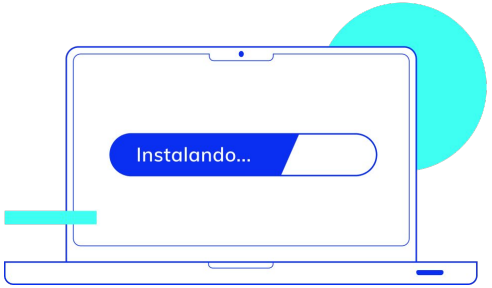
Este proceso consta de los siguientes pasos:



Ejemplo de paquete con Autools

Descarga del paquete

```
# curl -O https://dev.yorhel.nl/download/ncdu-1.14.2.tar.gz
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 141k 100 141k 0 0 67272 0 0:00:02 0:00:02 --:--:-- 67272
```



Descompresión y extracción del paquete

```
# tar xvzf ncdu-1.14.2.tar.gz
ncdu-1.14.2/
ncdu-1.14.2/install-sh
ncdu-1.14.2/Makefile.am
ncdu-1.14.2/deps/
ncdu-1.14.2/deps/khashl.h
ncdu-1.14.2/deps/yopt.h
ncdu-1.14.2/aclocal.m4
ncdu-1.14.2/Makefile.in
ncdu-1.14.2/doc/
ncdu-1.14.2/doc/ncdu.pod
ncdu-1.14.2/src/
ncdu-1.14.2/src/browser.c
ncdu-1.14.2/src/help.c
ncdu-1.14.2/src/dir.h
ncdu-1.14.2/src/quit.h
```

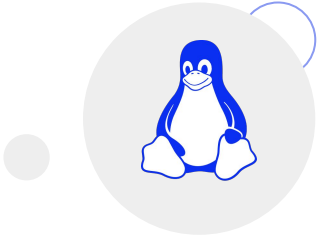
```
ncdu-1.14.2/src/delete.c
ncdu-1.14.2/src/dir_import.c
ncdu-1.14.2/src/dir_mem.c
ncdu-1.14.2/src/help.h
ncdu-1.14.2/src/delete.h
ncdu-1.14.2/src/exclude.c
ncdu-1.14.2/src/dir_scan.c
ncdu-1.14.2/src/dir_common.c
ncdu-1.14.2/src/path.c
ncdu-1.14.2/src/util.h
ncdu-1.14.2/src/path.h
ncdu-1.14.2/src/quit.c
ncdu-1.14.2/src/dir_export.c
ncdu-1.14.2/src/shell.h
ncdu-1.14.2/src/dirlist.h
ncdu-1.14.2/src/dirlist.c
```

```
ncdu-1.14.2/src/browser.h
ncdu-1.14.2/src/shell.c
ncdu-1.14.2/src/main.c
ncdu-1.14.2/src/exclude.h
ncdu-1.14.2/src/global.h
ncdu-1.14.2/src/util.c
ncdu-1.14.2/missing
ncdu-1.14.2/README
ncdu-1.14.2/COPYING
ncdu-1.14.2/configure.ac
ncdu-1.14.2/compile
ncdu-1.14.2/configure
ncdu-1.14.2/ChangeLog
ncdu-1.14.2/config.h.in
ncdu-1.14.2/depcomp
ncdu-1.14.2/ncdu.1
```

El paquete podría estar comprimido con `bzip2`, `gzip`, o `xz` como se ve en el *curso de Introducción a Linux*, de manera que hay que usar el modificador para el comando tar que corresponda. Si está comprimido con `zip` se puede usar el comando `unzip` para Linux.

Entrar en el directorio de las fuentes

```
# cd ncdu-1.14.2
```



Generación del archivo Makefile

En este paso se ejecuta el *script* **`configure`** que viene con el paquete este se encargará de verificar que está todo lo necesario para poder compilar e instalar el paquete, además va a generar un archivo llamado ***Makefile*** el cual tendrá una serie de recetas para el compilador:

```
# ./configure
./configure
checking for a BSD-compatible install... /usr/bin/install -c
checking whether build environment is sane... yes
checking for a thread-safe mkdir -p... /usr/bin/mkdir -p
checking for gawk... gawk
checking whether make sets $(MAKE)... yes
checking whether make supports nested variables... yes
checking for gcc... gcc
```

...

```
...
checking whether the C compiler works... yes
checking for C compiler default output file name... a.out
checking for suffix of executables...
checking whether we are cross compiling... no
checking for suffix of object files... o
checking whether we are using the GNU C compiler... yes
checking whether gcc accepts -g... yes
checking for gcc option to accept ISO C89... none needed
checking whether gcc understands -c and -o together... yes
checking whether make supports the include directive... yes (GNU style)
checking dependency style of gcc... gcc3
checking for ranlib... ranlib
checking for pkg-config... /usr/bin/pkg-config
checking pkg-config is at least version 0.9.0... yes
checking how to run the C preprocessor... gcc -E
checking for grep that handles long lines and -e... /usr/bin/grep
checking for egrep... /usr/bin/grep -E
checking for ANSI C header files... yes
...
```

```
...
checking for sys/types.h... yes
checking for sys/stat.h... yes
checking for stdlib.h... yes
checking for string.h... yes
checking for memory.h... yes
checking for strings.h... yes
checking for inttypes.h... yes
checking for stdint.h... yes
checking for unistd.h... yes
checking limits.h usability... yes
checking limits.h presence... yes
checking for limits.h... yes
checking sys/time.h usability... yes
checking sys/time.h presence... yes
checking for sys/time.h... yes
checking for sys/types.h... (cached) yes
checking for sys/stat.h... (cached) yes
checking dirent.h usability... yes
...
```

```
...
checking dirent.h presence... yes
checking for dirent.h... yes
checking for unistd.h... (cached) yes
checking fnmatch.h usability... yes
checking fnmatch.h presence... yes
checking for fnmatch.h... yes
checking ncurses.h usability... yes
checking ncurses.h presence... yes
checking for ncurses.h... yes
checking locale.h usability... yes
checking locale.h presence... yes
checking for locale.h... yes
checking for int64_t... yes
checking for uint64_t... yes
checking for special C compiler options needed for large files... no
checking for _FILE_OFFSET_BITS value needed for large files... no
checking for struct stat.st_blocks... yes
checking for getcwd... yes
...
```

```
...
checking for gettimeofday... yes
checking for fnmatch... yes
checking for chdir... yes
checking for rmdir... yes
checking for unlink... yes
checking for lstat... yes
checking for system... yes
checking for getenv... yes
checking for ncursesw... yes
configure: Using /bin/sh as the default shell if $SHELL is not set
checking that generated files are newer than configure... done
configure: creating ./config.status
config.status: creating Makefile
config.status: creating config.h
config.status: executing depfiles commands
...
```

Este paso es crucial, **si termina con error significa que no podrá compilarse el paquete.**

Las distribuciones suelen tener en sus repositorios compiladores y otras herramientas para desarrollo.

Por ejemplo

- En CentOS hay un grupo de paquetes que puede instalarse llamado “`Developement Tools`”.
- En Debian hay un metapaquete con el nombre “`build-essential`”.

Supongamos el siguiente escenario:

```
checking for unistd.h... yes
checking limits.h usability... yes
checking limits.h presence... yes
checking for limits.h... yes
checking sys/time.h usability... yes
checking sys/time.h presence... yes
checking for sys/time.h... yes
checking for sys/types.h... (cached) yes
checking for sys/stat.h... (cached) yes
checking dirent.h usability... yes
checking dirent.h presence... yes
checking for dirent.h... yes
checking for unistd.h... (cached) yes
checking fnmatch.h usability... yes
checking fnmatch.h presence... yes
checking for fnmatch.h... yes
checking ncurses.h usability... no
checking ncurses.h presence... no
checking for ncurses.h... no
configure: error: required header file not found
```

Vemos que el proceso terminó con **error**, en particular para los encabezados de la librería **`ncurses`**, por lo tanto en CentOS debería hacerse lo siguiente:

```
# yum install ncurses-devel
```

Y luego se deberá ejecutar nuevamente el *script* configure hasta que no haya más errores.

Es decir, en las distribuciones relacionadas con Red Hat los paquetes de desarrollo terminan con "**`-devel`**", mientras que en las distribuciones basadas en Debian finalizan con "**`-dev`**".

El *script* configure depende totalmente en la manera en que el desarrollador armó el paquete para compilar, pero en general posee una ayuda:

```
# ./configure --prefix
```

Por ejemplo, la opción **`--prefix`** define el directorio de instalación del paquete, en este caso se puede instalar como usuario común en algún directorio dentro de su **`$HOME`**.

El paso siguiente es efectivamente compilar:

```
# make
make  all-am
make[1]: Entering directory '/root/ncdu-1.14.2'
depbase=`echo src/browser.o | sed 's|[^/]*$|.deps/&|;s|\.o$||'`;\
gcc -DHAVE_CONFIG_H -I.  -I./deps   -g -O2 -MT src/browser.o -MD -MP -MF $depbase.Tpo -c -o src/browser.o
src/browser.c &&\
mv -f $depbase.Tpo $depbase.Po
depbase=`echo src/delete.o | sed 's|[^/]*$|.deps/&|;s|\.o$||'`;\
gcc -DHAVE_CONFIG_H -I.  -I./deps   -g -O2 -MT src/delete.o -MD -MP -MF $depbase.Tpo -c -o src/delete.o
src/delete.c &&\
mv -f $depbase.Tpo $depbase.Po
depbase=`echo src/dirlist.o | sed 's|[^/]*$|.deps/&|;s|\.o$||'`;\
gcc -DHAVE_CONFIG_H -I.  -I./deps   -g -O2 -MT src/dirlist.o -MD -MP -MF $depbase.Tpo -c -o src/dirlist.o
src/dirlist.c &&\
mv -f $depbase.Tpo $depbase.Po
depbase=`echo src/dir_common.o | sed 's|[^/]*$|.deps/&|;s|\.o$||'`;\
```

...

```
gcc -DHAVE_CONFIG_H -I.  -I./deps   -g -O2 -MT src/dir_common.o -MD -MP -MF $depbase.Tpo -c -o
src/dir_common.o src/dir_common.c &&\
mv -f $depbase.Tpo $depbase.Po
depbase=`echo src/dir_export.o | sed 's|[^/]*$|.deps/&|;s|\.o$||'`;\
gcc -DHAVE_CONFIG_H -I.  -I./deps   -g -O2 -MT src/dir_export.o -MD -MP -MF $depbase.Tpo -c -o
src/dir_export.o src/dir_export.c &&\
mv -f $depbase.Tpo $depbase.Po
depbase=`echo src/dir_import.o | sed 's|[^/]*$|.deps/&|;s|\.o$||'`;\
gcc -DHAVE_CONFIG_H -I.  -I./deps   -g -O2 -MT src/dir_import.o -MD -MP -MF $depbase.Tpo -c -o
src/dir_import.o src/dir_import.c &&\
mv -f $depbase.Tpo $depbase.Po
depbase=`echo src/dir_mem.o | sed 's|[^/]*$|.deps/&|;s|\.o$||'`;\
gcc -DHAVE_CONFIG_H -I.  -I./deps   -g -O2 -MT src/dir_mem.o -MD -MP -MF $depbase.Tpo -c -o src/dir_mem.o
src/dir_mem.c &&\
mv -f $depbase.Tpo $depbase.Po
depbase=`echo src/dir_scan.o | sed 's|[^/]*$|.deps/&|;s|\.o$||'`;\
gcc -DHAVE_CONFIG_H -I.  -I./deps   -g -O2 -MT src/dir_scan.o -MD -MP -MF $depbase.Tpo -c -o
src/dir_scan.o src/dir_scan.c &&\
```

```
mv -f $depbase.Tpo $depbase.Po
depbase=`echo src/exclude.o | sed 's|[^/]*$|.deps/&|;s|\.o$||'`;\
gcc -DHAVE_CONFIG_H -I.  -I./deps   -g -O2 -MT src/exclude.o -MD -MP -MF $depbase.Tpo -c -o src/exclude.o
src/exclude.c &&\
mv -f $depbase.Tpo $depbase.Po
depbase=`echo src/help.o | sed 's|[^/]*$|.deps/&|;s|\.o$||'`;\
gcc -DHAVE_CONFIG_H -I.  -I./deps   -g -O2 -MT src/help.o -MD -MP -MF $depbase.Tpo -c -o src/help.o
src/help.c &&\
mv -f $depbase.Tpo $depbase.Po
depbase=`echo src/shell.o | sed 's|[^/]*$|.deps/&|;s|\.o$||'`;\
gcc -DHAVE_CONFIG_H -I.  -I./deps   -g -O2 -MT src/shell.o -MD -MP -MF $depbase.Tpo -c -o src/shell.o
src/shell.c &&\
mv -f $depbase.Tpo $depbase.Po
depbase=`echo src/quit.o | sed 's|[^/]*$|.deps/&|;s|\.o$||'`;\
gcc -DHAVE_CONFIG_H -I.  -I./deps   -g -O2 -MT src/quit.o -MD -MP -MF $depbase.Tpo -c -o src/quit.o
src/quit.c &&\
mv -f $depbase.Tpo $depbase.Po
depbase=`echo src/main.o | sed 's|[^/]*$|.deps/&|;s|\.o$||'`;\
```

Y el último paso es instalar...

```
# make install
make[1]: Entering directory '/root/ncdu-1.14.2'
/usr/bin/mkdir -p '/usr/local/bin'
/usr/bin/install -c ncdu '/usr/local/bin'
/usr/bin/mkdir -p '/usr/local/share/man/man1'
/usr/bin/install -c -m 644 ncdu.1 '/usr/local/share/man/man1'
make[1]: Leaving directory '/root/ncdu-1.14.2'
```

Si quisiéramos desinstalar se debería ejecutar:

```
# make uninstall
( cd '/usr/local/bin' && rm -f ncdu )
( cd '/usr/local/share/man/man1' && rm -f ncdu.1 )
```

Compilar usando CMake

`CMake` se usa mucho con los lenguajes C y C++, y se usa en proyectos de software libre (Ninja), software privativo (Visual Studio) y mixto (Xcode).

Hay algunos paquetes que usan un tipo de **controlador de compilación** distinto como **`cmake`**. En este caso el tipo de compilación es un poco distinto.

En general, se prefiere no compilar sobre el propio directorio de las fuentes sino en un directorio aparte.

```
$ mkdir build && cd build
```

Luego, creamos los archivos necesarios para poder compilar

```
$ cmake -DCMAKE_BUILD_TYPE=release ..
```

Notar como apuntamos al directorio de las fuentes. La opción **`-DCMAKE_BUILD_TYPE=release`** en este caso puntual es para indicarle el tipo de compilación.

Si se presenta un error como vemos a continuación:

```
-- Looking for GnuTLS
-- Could NOT find GnuTLS (missing: GNUTLS_LIBRARY GNUTLS_INCLUDE_DIR)
CMake Error at CMakeLists.txt:127 (message):
 Cannot find GnuTLS.  Use -DENABLE_SYNC=OFF to build Taskwarrior without
 sync support.  See INSTALL for more information.
```

Eso indica que un archivo necesario para compilar con todas las funcionalidades está ausente. En Debian 10, lo solucionaremos así:

```
# apt -y install libgnutls28-dev
```

Luego ejecutamos la herramienta de compilación:

```
# cmake --build
```

Y finalmente instalamos el paquete compilado:

```
# make install
```

```
gcc -DHAVE_CONFIG_H -I.  -I./deps   -g -O2 -MT src/main.o -MD -MP -MF $depbase.Tpo -c -o src/main.o
src/main.c &&\
mv -f $depbase.Tpo $depbase.Po
depbase=`echo src/path.o | sed 's|[^/]*$|.deps/&|;s|\.o$||'`;\
gcc -DHAVE_CONFIG_H -I.  -I./deps   -g -O2 -MT src/path.o -MD -MP -MF $depbase.Tpo -c -o src/path.o
src/path.c &&\
mv -f $depbase.Tpo $depbase.Po
depbase=`echo src/util.o | sed 's|[^/]*$|.deps/&|;s|\.o$||'`;\
gcc -DHAVE_CONFIG_H -I.  -I./deps   -g -O2 -MT src/util.o -MD -MP -MF $depbase.Tpo -c -o src/util.o
src/util.c &&\
mv -f $depbase.Tpo $depbase.Po
gcc  -g -O2   -o ncdu src/browser.o src/delete.o src/dirlist.o src/dir_common.o src/dir_export.o
src/dir_import.o src/dir_mem.o src/dir_scan.o src/exclude.o src/help.o src/shell.o src/quit.o src/main.o
src/path.o src/util.o  -lncursesw -ltinfo
make[1]: Leaving directory '/root/ncdu-1.14.2'
```

educación IT
CODE YOUR FUTURE

¡Sigamos trabajando!